설비 고장을 예측하는 AI 데이터 분석

NumPy 실습

센서 데이터 구조화와 배열 응용

posco

CONTENTS

목차

01 위치와 변화량 찾기 — argmax · argsort · diff · clip

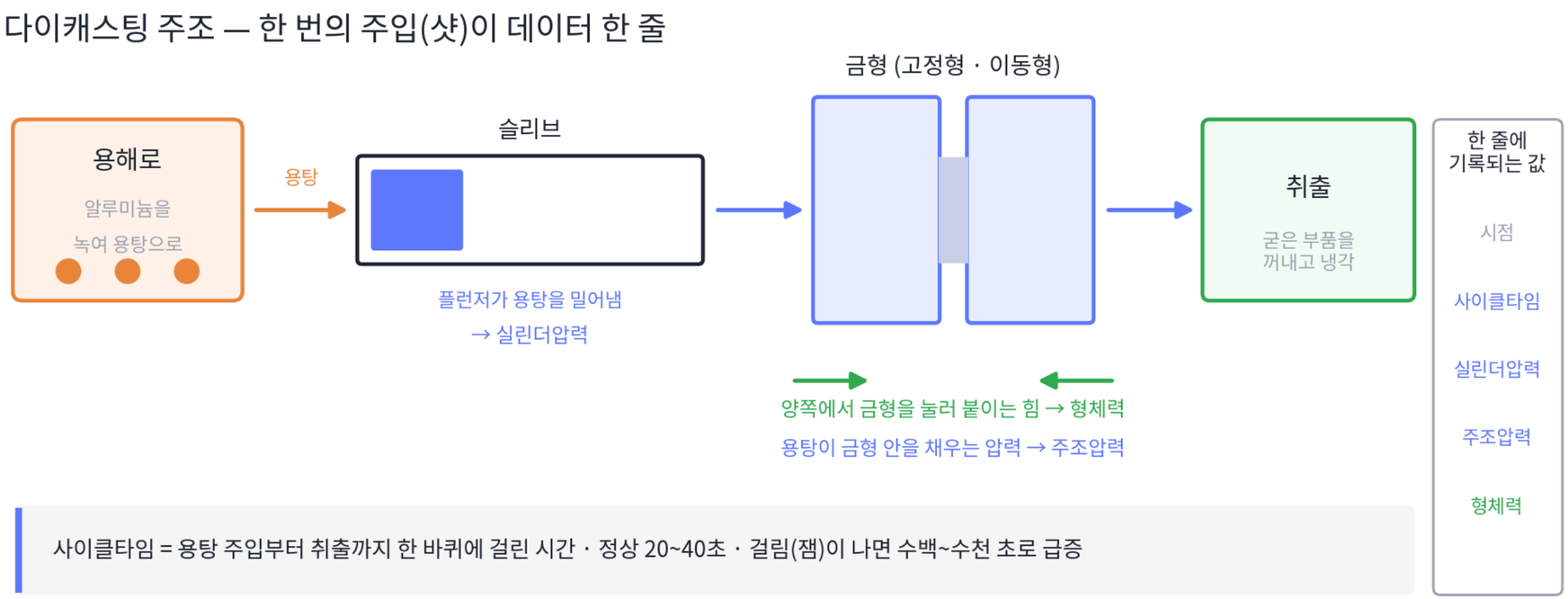
02 상태 라벨링과 배열 합치기 — where · unique · stack

03 종합 정리 — 도구를 하나의 흐름으로

posco

분석 대상 공정 — 다이캐스팅 주조

녹인 금속을 금형에 밀어 넣어 부품을 만드는 설비 · 한 번의 주입을 샷이라 부름



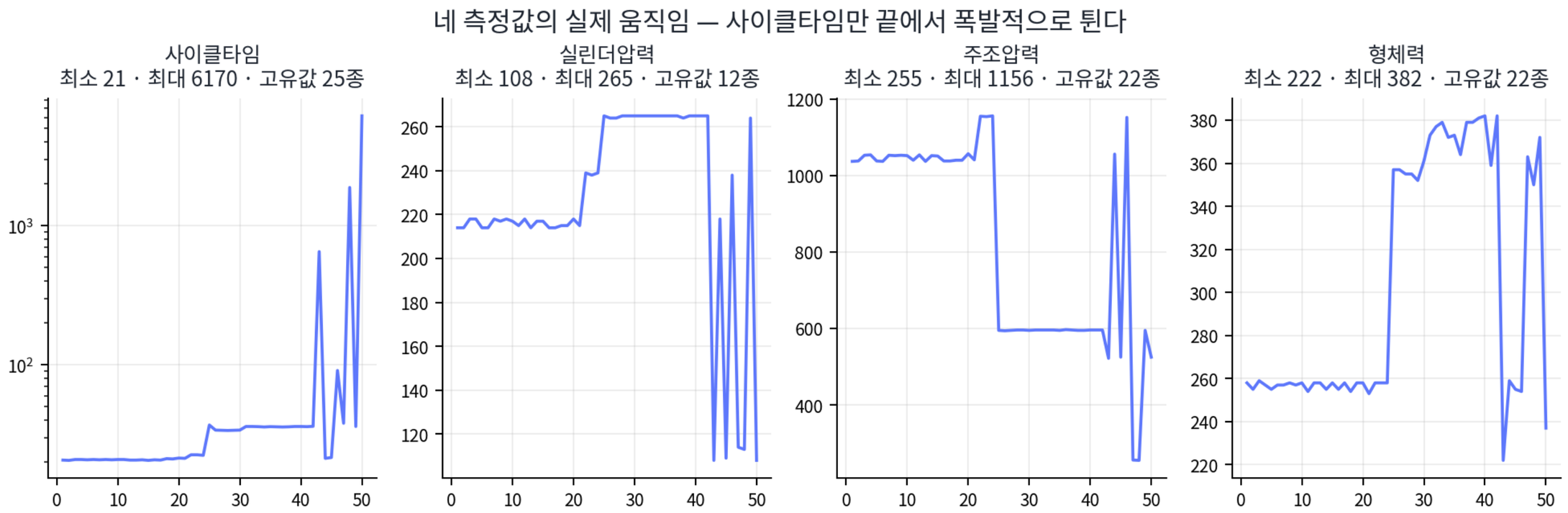
용해로에서 녹인 금속을 슬리브에 붓고, 플런저가 그것을 금형 안으로 밀어 넣음

한 번의 샷이 끝날 때마다 사이클타임·실린더압력·주조압력·형체력이 한 줄로 기록됨

출처: 주조(다이캐스팅) 설비 실데이터(KAMP 제조AI 데이터셋) 발췌 · 50샷 5열

네 측정값이 실제로 어떻게 움직이는가

무엇을 재는 값인지 알아야 결과를 해석할 수 있음 · 네 열의 실측 추이



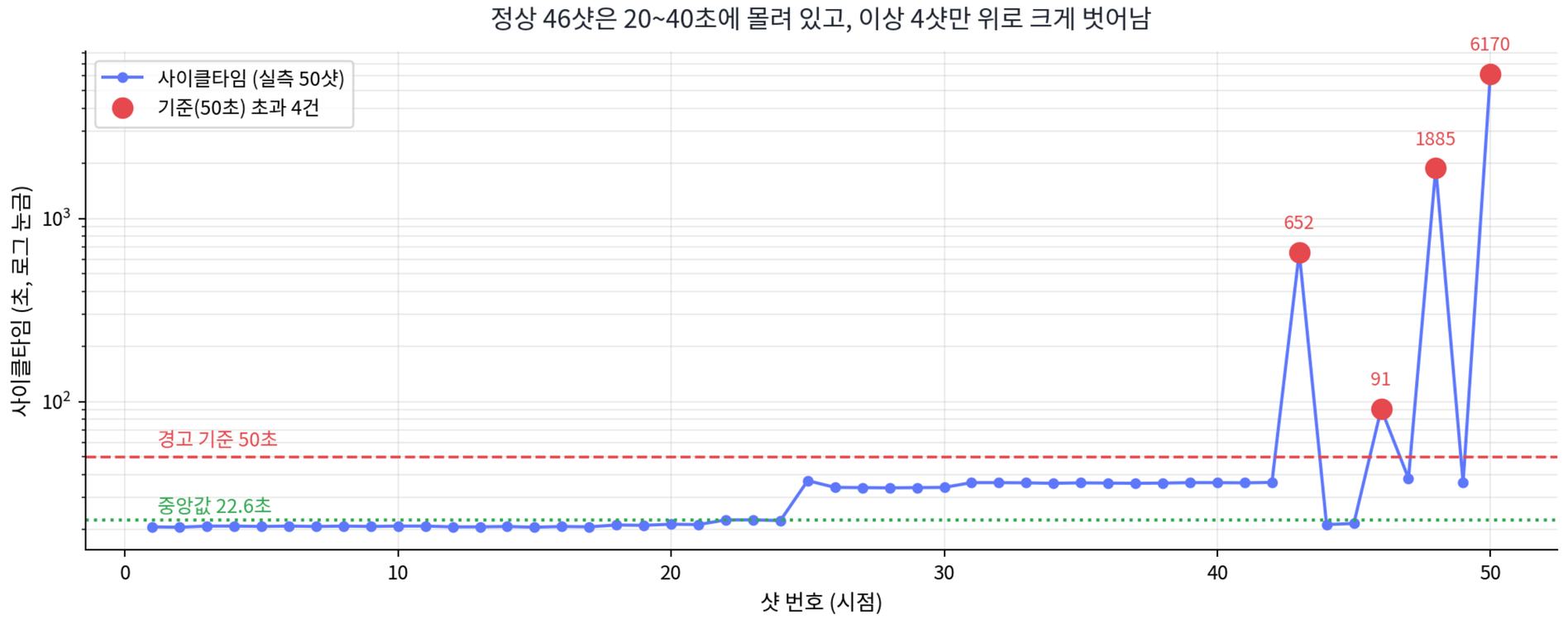
사이클타임만 끝부분에서 수천 배로 튐 — 나머지 세 값은 두 배 안쪽에서 오르내림

그래서 이상 판정의 주 지표는 사이클타임, 실린더압력은 함께 보는 보조 지표

실린더압력은 12종, 주조압력·형체력은 22종의 값만 나타남 — 설비가 몇 가지 조건으로만 운전됐다는 뜻

정상 샷과 잼 — 무엇이 다른가

정상은 20~40초로 일정 · 걸림이 나면 수백~수천 초로 급증 · 50샷 중 4건



46샷은 중앙값 22.6초 부근에 몰려 있고, 4샷만 기준선 50초 위로 크게 벗어남

50샷이면 눈으로 셀 수 있지만 하루치는 수천 샷 — 그래서 코드로 찾아야 함

기준 50초는 데이터에서 나온 값이 아니라 현장에서 정한 값 · 설비를 아는 사람이 알려줘야 정할 수 있음

posco

배열 생성·속성

오늘 종일 쓸 도구라 손이 기억하게 다시 한 번 정리

01 · 배열 생성

리스트→배열 변환과 연속·균등·빈 배열

np.array로 리스트를 배열로
arange는 간격을 정해 값 생성
linspace는 같은 간격의 값 N개
zeros · ones는 0 · 1로 채운 배열

02 · 배열 속성

배열의 모양·차원 자료형 확인과 변환

shape는 모양
ndim은 차원 수
dtype은 자료형
astype으로 형 변환

03 · 벡터 연산

반복문 없이 한 줄로 전체 연산

반복문 없이 배열 전체에
한 번에 **사칙연산** 적용

데이터가 클수록 차이 극대화

수만 개 보정 한 줄로 끝나는 벡터 연산이 NumPy의 진짜 강점

posco

인덱싱·슬라이싱·조건 필터링

이상 센서값을 찾아내는 모든 작업의 출발점

01 · 인덱싱

위치 번호로
값 하나 꺼내기

대괄호에 위치 번호를 넣어 값 하나 꺼내기

0부터 시작, **-1**은 맨 뒤

02 · 슬라이싱

콜론으로
연속 구간 잘라내기

콜론으로 시작과 끝을 정해

연속 구간을 한 번에 잘라내기

03 · 불리언 인덱싱

True · False로
원하는 값만 골라내기

조건으로 만든 **True · False 배열** 로

원하는 값만 골라내기

불리언 인덱싱이 오늘 가장 자주 등장할 도구

posco

센서 분석의 4단계 흐름

담기 → 꺼내기 → 요약 → 판정, 네 단계 흐름

STEP 1

담기

센서값을 배열에 담는 단계

어제 배운 배열 만들기

→

STEP 2

꺼내기

원하는 부분을 꺼내고 조건으로 골라내는 단계

→

STEP 3

요약

수천 개 값을 평균·표준편차 같은 몇 개 숫자로 압축

→

STEP 4

판정

요약한 통계를 기준으로 이상한 값을 가려냄

argmax · diff · where

오늘은 담기와 꺼내기까지, 요약과 판정은 다음 단원에서

posco

파일에서 배열로 불러오기(np.genfromtxt)

앞 단원의 loadtxt와 같은 일을 하되, 빈 칸을 만나도 멈추지 않고 값 없음으로 채움

CODE

```
import numpy as np

F = 'data/11_diecasting_series.csv'

ct = np.genfromtxt(F,
                   delimiter=',',     # 값을 나누는 기호
                   skip_header=1,     # 첫 줄(제목)은 건너뜀
                   usecols=1)         # 2번째 열 = 사이클타임
print(ct.shape, ct.dtype)
# 출력: (50,) float64

# 열 번호는 0부터
# 시점 0 · 사이클타임 1 · 실린더압력 2 · 주조압력 3 · 형체력 4
```

loadtxt는 건너뛸 줄을 skiprows, genfromtxt는 skip_header로 씀 · 열 번호는 둘 다 0부터

posco

01

위치와 변화량 찾기

argmax · argsort · diff · clip · cumsum

값이 아니라 자리를 찾는다

앞 단원은 「얼마인가」에 답했고, 이 단원은 「어디서 그랬는가」에 답한다

01 · 앞 단원에서 한 일

가장 큰 값은 얼마인가

최댓값 6170.0 을 구했음. 값 자체는 알았지만 그것이 몇 번째 샷이었는지는 따로 눈으로 찾아야 했음.

02 · 이 단원에서 할 일

그 값이 어디서 났는가

50번 샷이라는 답을 바로 얻음. 설비 점검은 「언제 무슨 일이 있었나」를 알아야 시작할 수 있음.

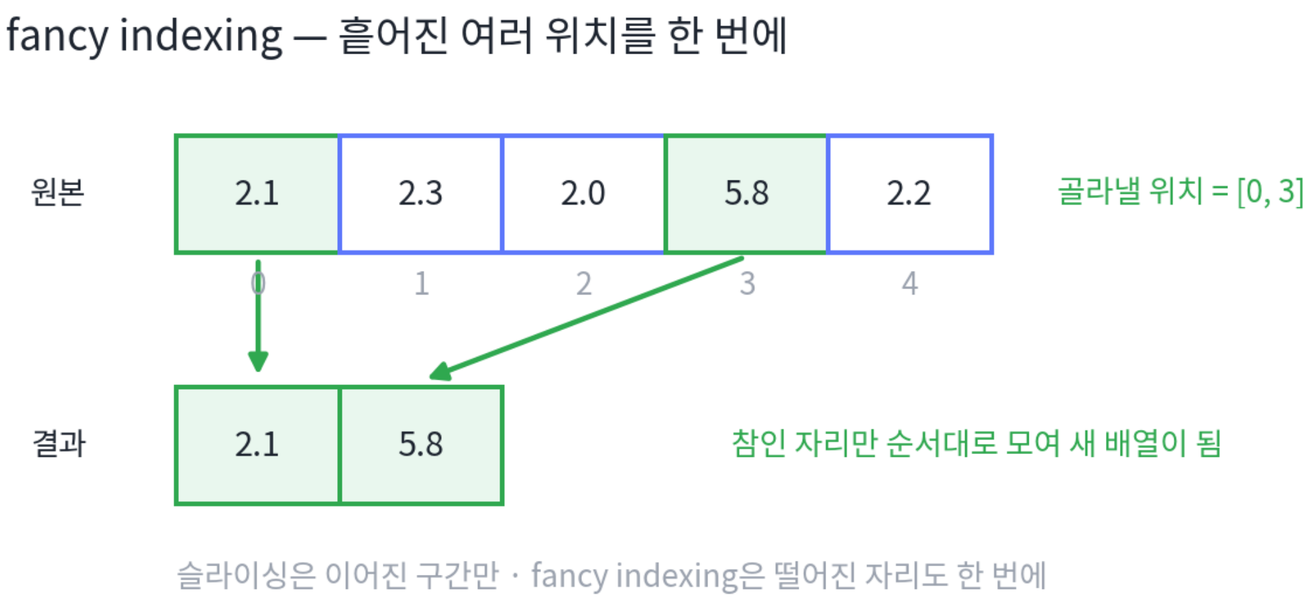
03 · 자리를 알면

앞뒤를 함께 볼 수 있음

자리를 알면 그 시점 앞뒤의 다른 센서값도 같이 꺼내 볼 수 있음. 원인을 찾는 출발점이 여기서 만들어짐.

여러 위치 한 번에 추출 — fancy indexing

흩어진 여러 자리의 값을 한 번에 꺼내는 도구



꺼내고 싶은 자리 번호를 묶음으로 만들어 대괄호에 넣으면 그 자리 값만 모인 새 배열이 생김

슬라이싱은 이어진 구간만 잘라낼 수 있지만, 이 방법은 떨어진 자리도 한 번에 꺼냄

결과는 원본과 분리된 새 배열 — 결과를 고쳐도 원본은 그대로

posco

fancy indexing — 핵심 코드

위치 목록을 리스트로 묶어 추출

CODE

```
vib = np.array([2.1, 2.3, 2.0, 5.8])
pick = np.array([0, 3])
print(vib[pick])
# 출력: [2.1 5.8]
```

posco

정렬과 정렬 순서 — sort, argsort

값을 늘어놓은 결과와 그 값들이 원래 있던 자리 — 두 가지 정보가 따로 필요



sort는 값을 작은 것부터 늘어놓고, argsort는 그 순서에 해당하는 원래 자리 번호를 돌려줌

「가장 큰 값이 얼마인가」가 아니라 「언제 그랬는가」를 물을 때는 자리 번호가 필요

상위 세 개의 자리를 알고 싶으면 argsort 결과의 맨 뒤 세 개

posco

sort, argsort — 핵심 코드

큰 값 상위 2개의 위치를 argsort로 추출

CODE

```
import numpy as np
vib = np.array([2.1, 6.1, 2.0, 5.8])
print(np.sort(vib))
# 출력: [2. 2.1 5.8 6.1]
print(np.argsort(vib))
# 출력: [2 0 3 1]
```

전체 동작 코드는

posco

최대·최소 발생 위치 — argmax, argmin

이상 분석의 첫 질문은 '가장 심했던 순간이 언제인가'

이상 분석의 첫 질문은 **'가장 심했던 순간이 언제인가'**

np.max는 값을, **np.argmax**는 그 값의 위치를 반환

찾은 위치를 사이클·온도 배열에 넣어 그 순간의 다른 정보까지 연결

> 이상 탐지에서 **'어디서'는 조치의 출발점**. 실무는 max보다 argmax가 빈번

posco

argmax, argmin — 핵심 코드

가장 큰 진동값의 위치를 찾아 사이클로 변환

CODE

```
import numpy as np
cycle = np.array([1, 2, 3, 4, 5])
vib = np.array([2.1, 2.3, 6.1, 2.0, 5.8])
pos = np.argmax(vib)
print(cycle[pos])
# 출력: 3
```

posco

실습 1. 최대 실린더 압력 시점 찾기

실린더 압력이 가장 높은 샷이 몇 번째였는지 찾아 값과 함께 확인

목표

실린더 압력이 가장 컸던 시점을 찾아 사이클 번호와 값을 함께 출력

단계

① 공정 데이터 파일에서 시점과 실린더 압력 두 열을 각각 불러오기

② 실린더 압력 묶음에서 가장 큰 값이 몇 번째 자리에 있는지 찾기

③ 그 자리 번호를 시점 묶음에 넣어 실제 사이클 번호로 바꾸기

④ 같은 자리 번호를 실린더 압력 묶음에도 넣어 그때의 값을 꺼내기

⑤ 실린더 압력 번호와 값을 함께 출력해 정말 이상한 값인지 확인

예상 결과

실린더 압력 중앙값은 ###인데, n번째 사이클에서 n값이 나옴

실습 1. 최대 사이클타임 시점 찾기 결과 해석

찾아낸 자리 번호 하나가 다른 모든 센서로 이어짐

01 · 왜 위치인가

값만 알면 언제인지 모름

6170.0초라는 값만으로는 정비 이력을 확인할 수 없음. 몇 번째 샷이었는지 알아야 그 시각의 작업 기록·교대조·금형 상태를 찾아볼 수 있음.

02 · 자리 번호의 힘

한 번호로 그 순간 전체를 봄

시점·사이클타임·압력이 모두 같은 순서로 저장되어 있음. 그래서 자리 번호 하나만 있으면 그 순간의 다른 센서값도 전부 꺼낼 수 있음.

03 · 검산

중앙값의 273배

사이클타임 중앙값은 22.6초. 찾아낸 6170.0초는 그 273배임. 기준을 따로 정하지 않아도 한눈에 이상하다고 말할 수 있는 크기.

다음 질문은 「언제부터 그랬나」 — 값이 아니라 변화를 봐야 답이 나옴

posco

실습 1. 최대 사이클타임 시점 찾기 정답

자리 번호를 구해 두면 다른 배열에도 그대로 넣어 쓸 수 있다

CODE

```
import numpy as np

# 출처: 주조(다이캐스팅) 설비 실데이터(KAMP) 발췌
F = 'data/11_diecasting_series.csv'
cyc = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=0)
ct  = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=1)

i = ct.argmax()
print(int(cyc[i]), ct[i])   # 50 6170.0
```

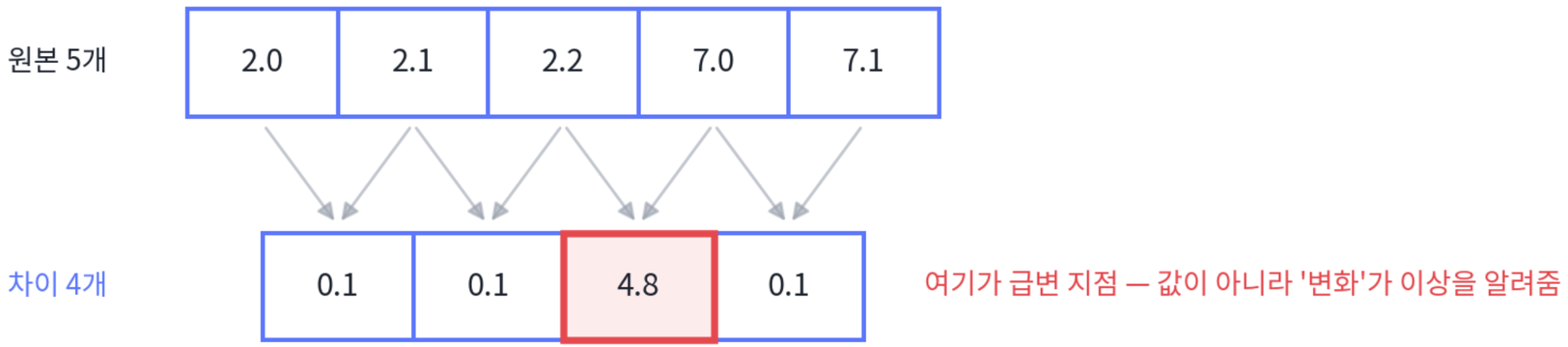
argmax가 돌려주는 것은 값이 아니라 자리 번호 — 값을 보려면 배열에 다시 넣어야 함

posco

인접 시점 변화량 — diff

값 자체보다 「얼마나 갑자기 변했나」가 더 강한 이상 신호

np.diff — 이웃한 값의 차이. 결과 길이가 하나 줄어든다



값 5개 → 차이 4개. 차이는 이웃 사이에서 생기므로 항상 하나가 짧다

이웃한 두 값의 차이를 구하므로 결과 개수는 원본보다 항상 하나 적음

k번째 변화량은 k번과 k+1번 사이에서 생긴 값 — 한 시점이 아니라 구간을 가리킴

급증도 급감도 똑같이 이상이므로 부호를 없앤 뒤 크기만 비교

posco

diff — 핵심 코드

변화량을 구하고 가장 급증한 위치 찾기

CODE

```
import numpy as np
vib = np.array([2.0, 2.1, 2.2, 7.0, 7.1])
change = np.diff(vib)
print(np.round(change, 1))
# 출력: [0.1 0.1 4.8 0.1]
print(np.argmax(change))
# 출력: 2
```

diff 결과는 원본보다 1 짧음

센서값 정리 — round, abs

이상 탐지는 방향 없이 거리만 보므로 abs가 결정적

np.round는 소수점 반올림, 보고·표시용 마지막 단계에서 사용

np.abs는 부호를 없애 변화의 크기만 비교

이상 탐지는 위·아래 어느 방향이든 거리만 보므로 abs가 중요

> 내일 z-score 이상 탐지에서 abs 재등장. **평균에서 떨어진 거리**를 잴 때 필수

round는 중간이 아니라 사람에게 보여주는 마지막 단계에서

posco

round, abs — 핵심 코드

반올림과 절댓값으로 정리

CODE

```
import numpy as np
print(np.round(2.6666666, 2))
# 출력: 2.67

change = np.array([0.1, -4.8, 4.8, -0.1])
print(np.abs(change))
# 출력: [0.1 4.8 4.8 0.1]
```

변화량의 크기만 비교할 땐 abs가 핵심

posco

실습 2. 사이클타임 급변 구간 찾기

이웃한 두 시점 사이에서 값이 가장 크게 변한 구간을 찾기

목표

값이 가장 급격하게 변한 구간의 앞뒤 사이클 번호와 변화량을 출력

단계

① 시점과 사이클타임 두 열을 불러오기

② 이웃한 값끼리의 차이를 구해 변화량 묶음 만들기

③ 오르내림을 구분하지 않도록 변화량의 부호를 없애기

④ 부호를 없앤 변화량에서 가장 큰 값의 자리를 찾기

⑤ 그 자리와 바로 다음 자리의 시점을 함께 출력

예상 결과

49번과 50번 사이에서 6133.9초만큼 변한 것으로 출력

실습 2. 사이클타임 급변 구간 찾기 결과 해석

값이 아니라 변화를 보면 「언제부터」에 답할 수 있음

01 · 하나 짧은 이유

차이는 사이 공간에서 생김

값 50개 사이의 공간은 49개. 그래서 변화량도 49개가 됨. 자리 번호는 한 시점이 아니라 그 자리와 다음 자리 사이의 구간을 가리킴.

02 · 부호를 없애는 이유

급감도 똑같이 이상

값이 갑자기 떨어지는 것도 설비 이상. 부호를 그대로 두면 급감은 음수라서 가장 큰 값을 찾을 때 밀려남. 절댓값을 씌워야 둘 다 잡힘.

03 · 검산

36.1 → 6170.0

49번 사이클타임은 36.1초, 50번은 6170.0초. 직접 빼면 6133.9. 실습 1이 찾아낸 50번과 이어지는 결과임.

실습 1은 「어디가 가장 심한가」, 실습 2는 「어디서 급변했나」 — 두 답이 같은 구간을 가리킴

posco

실습 2. 사이클타임 급변 구간 찾기 정답

변화량의 자리 번호는 앞뒤 두 시점을 함께 짚어야 구간이 된다

CODE

```
import numpy as np

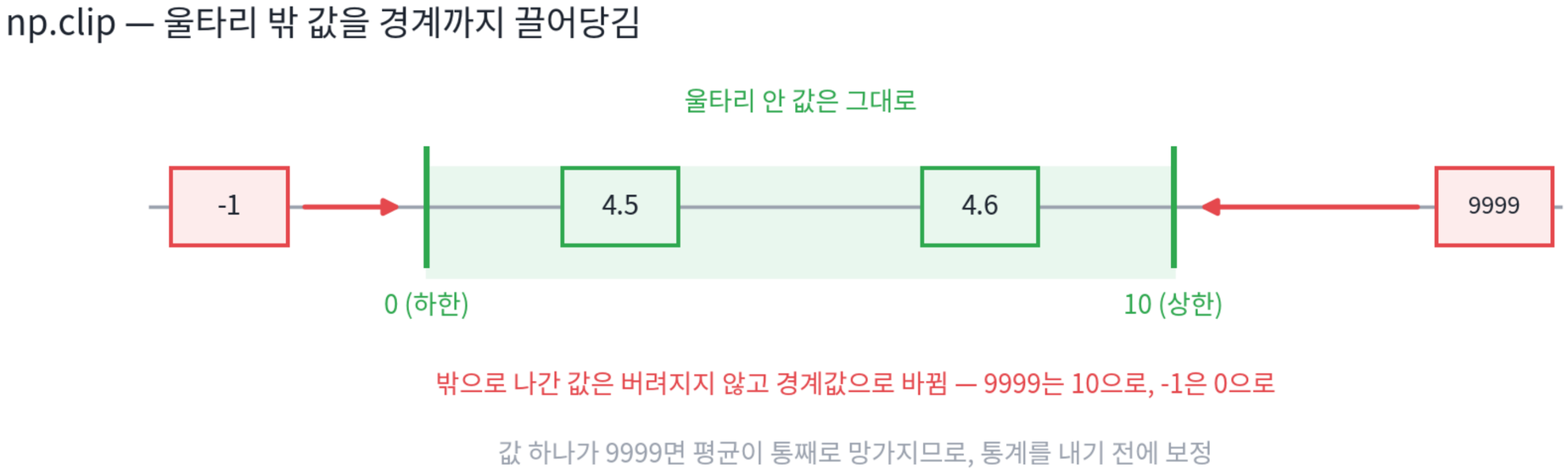
# 출처: 주조(다이캐스팅) 설비 실데이터(KAMP) 발췌
F = 'data/11_diecasting_series.csv'
cyc = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=0)
ct  = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=1)

change = np.abs(np.diff(ct))
j = change.argmax()
print(int(cyc[j]), int(cyc[j + 1]), round(change[j], 1))   # 49 50 6133.9
```

diff 결과는 원본보다 하나 짧음 — 앞뒤 두 시점을 함께 출력해야 어느 구간인지 드러남

센서값 범위 제한 보정 — clip

측정 오류로 튄 값은 눌러 두고 진짜 이상은 살려 두기



정해 둔 아래·위 경계를 벗어난 값은 버려지지 않고 경계값으로 바꿈

값 하나가 9999면 평균이 통째로 망가지므로 통계를 내기 전에 먼저 보정

경계를 어디에 둘지는 데이터가 아니라 그 설비를 아는 사람이 정하는 값

posco

clip — 핵심 코드

0~10 범위를 벗어난 값을 경계로 보정

CODE

```
import numpy as np
pres = np.array([4.5, 9999.0, 4.6, -1.0])
fixed = np.clip(pres, 0, 10)
print(fixed)
# 출력: [4.5 10. 4.6 0. ]
```

9999는 10으로, 음수는 0으로 보정

보정과 제거 — 어느 쪽을 쓸까

튄 값을 눌러 담을 것인가, 아예 빼 버릴 것인가

01 · 눌러 담기

줄 수가 그대로

경계까지 눌러 담으면 50샷이 50샷 그대로 남음. 시점이 어긋나지 않으므로 앞뒤 비교가 그대로 가능.

02 · 빼 버리기

줄 수가 줄어듦

튄 값을 빼면 그 시점이 통째로 사라져 앞뒤가 붙어 버림. 순서가 있는 데이터에서는 특히 조심해야 함.

03 · 판단 기준

측정 오류인가 실제 사건인가

센서가 잘못 읽어 튄 값이면 눌러 담아도 됨. 잼처럼 실제로 일어난 일이면 지우면 안 되고 표시해 두어야 함.

posco

실습 3. 측정 오류값 범위 안으로 보정하기

범위를 벗어난 값 하나가 평균을 얼마나 망가뜨리는지 숫자로 확인

목표

오류가 섞인 상태와 보정한 뒤의 평균을 나란히 놓고 차이를 확인

단계

① 실린더압력 열을 불러오기

② 첫 번째 값을 일부러 9999로 바꿔 측정 오류 상황을 만들기

③ 오류가 섞인 상태의 평균을 구해 출력

④ 값을 100과 300 사이로 눌러 보정하기

⑤ 보정한 뒤의 평균을 출력해 원래 수준으로 돌아오는지 확인

예상 결과

보정 전 평균은 421.5, 보정 뒤에는 227.5로 원래 수준 부근

실습 3. 측정 오류값 범위 안으로 보정하기 결과 해석

정제를 먼저 하지 않으면 그 뒤의 통계는 전부 어긋남

01 · 값 하나의 위력

평균이 225.8에서 421.5로

50개 중 단 하나를 9999로 바꿨을 뿐인데 평균이 두 배 가까이 움직임. 평균은 모든 값을 더해 나누므로 극단값 하나에 그대로 끌려감.

02 · 완전 복구는 아님

보정 뒤 227.5

원래 평균 225.8과 정확히 같지는 않음. 9999가 사라진 것이 아니라 경계값 300으로 눌렸기 때문. 보정은 원상 복구가 아니라 피해를 줄이는 일.

03 · 경계는 누가 정하나

데이터가 아니라 현장 지식

100과 300은 데이터에서 계산해 나온 값이 아님. 「이 설비의 실린더압력은 이 범위를 벗어날 수 없다」는 사실을 아는 사람이 정해 주는 값.

posco

실습 3. 측정 오류값 범위 안으로 보정하기 정답

보정 전후를 나란히 출력해야 무엇이 얼마나 달라졌는지 보인다

CODE

```
import numpy as np

# 출처: 주조(다이캐스팅) 설비 실데이터(KAMP) 발췌
F = 'data/11_diecasting_series.csv'
cyl = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=2)

cyl[0] = 9999                    # 측정 오류가 하나 섞인 상황
print(round(cyl.mean(), 1))      # 421.5

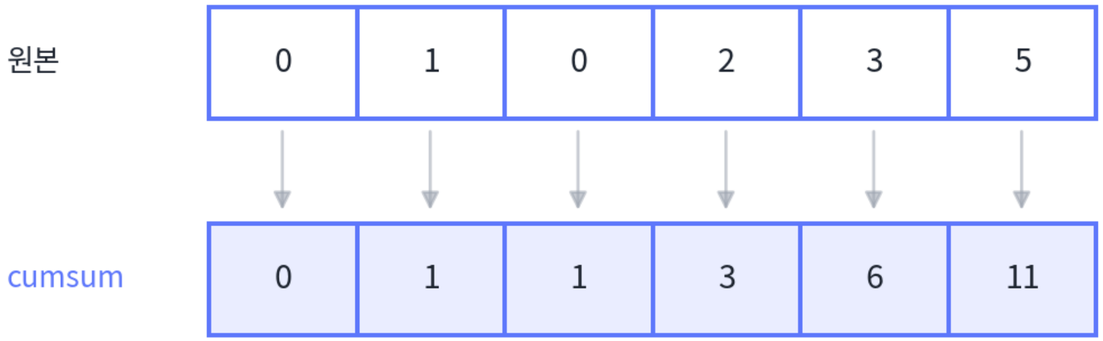
fixed = np.clip(cyl, 100, 300)
print(round(fixed.mean(), 1))    # 227.5
```

원래 평균은 225.8 — 보정 뒤 227.5는 완전 복구가 아니라 피해를 줄인 결과

누적 합으로 추세 보기 — cumsum

순간값에 가려진 전체 흐름은 누적 곡선에서 매끄럽게 드러남

np.cumsum — 앞에서부터 더해 온 합계를 자리마다 남김



0 · 0+1 · 0+1+0 · … 처럼 앞쪽 값이 계속 누적됨. 누적 불량 수, 누적 가동 시간처럼 '지금까지 합계'를 볼 때 사용

자리마다 「거기까지의 합계」를 남기므로 결과 개수는 원본과 같음

누적 불량 수, 누적 가동 시간처럼 「지금까지 얼마나 쌓였나」를 볼 때 사용

값이 오르내려도 누적 곡선은 항상 오르거나 유지 — 그래서 추세가 눈에 잘 들어옴

posco

cumsum — 핵심 코드

사이클별 불량 개수의 누적 합

CODE

```
import numpy as np
defects = np.array([0, 1, 0, 2, 3, 5])
print(np.cumsum(defects))
# 출력: [0 1 1 3 6 11]
```

sum은 최종 합 하나, cumsum은 매 시점 누적값 전체

posco

위치·변화량 추적 흐름

센서값 → 변화량 → 크기 → 위치 → 적용, 다섯 단계 흐름

STEP 1

센서값

센서값 배열에서 시작.
진동·온도·압력 등
원본 측정값

`np.array`

→

STEP 2

변화량

diff로 인접 변화량을 구함

`np.diff`

→

STEP 3

크기

abs로 부호를 없애
크기만 남김

`np.abs`

→

STEP 4

위치

argmax로
가장 크게 변한 위치 탐색

`np.argmax`

→

STEP 5

적용

그 위치를
사이클·타 센서에 적용해
입체적 파악

`fancy indexing`

위치를 알면 사이클·다른 센서의 그때 상태까지 한 번에 연결

posco

02

상태 라벨링과 배열 합치기

where · unique · any · all · concatenate · stack

posco

다중 조건 라벨링 — np.where 심화

where의 진짜 힘은 판단 자동화

측정값을 **정상·경고 같은 상태 라벨로 자동 변환**

조건이 참이면 한 값, 거짓이면 다른 값을 채움

where를 겹쳐 쓰면 **3단계 이상 다단계 라벨링** 가능

> 기준을 바꾸고 싶으면 **코드의 숫자 하나만 고치면 전체가 다시 계산**

수천 개 측정값이 단번에 0과 1로 — 사람 판단 불필요

posco

np.where 심화 — 핵심 코드

3단계 자동 라벨링 (5이상=2, 3이상=1, 그외=0)

CODE

```
import numpy as np
vib = np.array([2.1, 3.5, 2.0, 6.2])
label = np.where(vib >= 5, 2, np.where(vib >= 3, 1, 0))
print(label)
# 출력: [0 1 0 2]
```

바깥 where의 거짓 자리에 또 다른 where

posco

상태 라벨 종류 세기 — unique

> **분석 첫 단계는 '데이터에 어떤 상태가, 몇 개씩 있나' 확인**

분석 첫 단계는 '데이터에 어떤 상태가, 몇 개씩 있나' 확인

np.unique는 값의 종류를 중복 없이 정리

return_counts 옵션으로 각 종류의 개수까지 함께 확인

> 설비 데이터에서 고장은 매우 드묾. **불균형을 모르면 엉뚱한 결론**

정상이 990개, 고장이 10개라면 크게 치우친 데이터

posco

unique — 핵심 코드

상태 종류와 개수를 한 번에 집계

CODE

```
import numpy as np
status = np.array([0, 0, 1, 0, 2, 1, 0])
kinds, counts = np.unique(status, return_counts=True)
print(kinds)
print(counts)
# 출력: [0 1 2] / [4 2 1]
```

return_counts를 켜야 개수가 함께 출력

posco

실습 4. 상태를 세 단계로 나누어 세기

사이클타임을 정상·경고·잼 세 단계로 나누고 각 단계가 몇 건인지 세기

목표

세 단계로 나눈 뒤 단계별 개수와 전체 대비 비율을 출력

단계

① 사이클타임 열을 불러오기

② 1000 이상이면 잼, 50 이상이면 경고, 나머지는 정상이 되도록 세 단계 번호 매기기

③ 매긴 번호가 자리마다 몇 개씩 있는지 세기

④ 세 개수를 전체 개수로 나눠 비율 구하기

⑤ 개수와 비율을 함께 출력해 경고 이상이 전체의 몇 할인지 확인

예상 결과

정상 46 · 경고 2 · 잼 2, 비율은 0.92 · 0.04 · 0.04

실습 4. 상태를 세 단계로 나누어 세기 결과 해석

숫자를 등급으로 바꾸는 순간 「경험상」이 「전체의 몇 %」가 됨

01 · 순서가 결과를 바꿈

큰 기준부터 걸러야 함

50 이상을 먼저 판단하면 6170도 경고로 분류되어 잼이 하나도 남지 않음. 조건을 겹쳐 쓸 때는 항상 큰 기준을 바깥에 두어야 함.

02 · 종류를 먼저 확인

없는 라벨은 안 나옴

잼이 한 건도 없는 데이터라면 종류가 두 가지만 나옴. 개수만 보지 말고 어떤 종류가 나왔는지 함께 확인해야 자리를 잘못 읽지 않음.

03 · 검산

46 + 2 + 2 = 50

단계별 개수를 더하면 전체 샷 수와 같아야 함. 비율도 0.92 + 0.04 + 0.04 = 1.00. 둘 다 맞아야 라벨링이 새지 않은 것.

실습 4. 상태를 세 단계로 나누어 세기 정답

조건을 겹쳐 쓸 때는 큰 기준을 바깥에 둔다

CODE

```
import numpy as np

# 출처: 주조(다이캐스팅) 설비 실데이터(KAMP) 발췌
F = 'data/11_diecasting_series.csv'
ct = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=1)

label = np.where(ct >= 1000, 2, np.where(ct >= 50, 1, 0))

kinds, count = np.unique(label, return_counts=True)
print(kinds, count)              # [0 1 2] [46  2  2]
print(count / count.sum())       # [0.92 0.04 0.04]
```

주의) 잼이 하나도 없는 데이터가 오면 2가 아예 안 나옴 — 종류를 먼저 확인해야 하는 이유

posco

조건 충족 여부 판정 — any, all

현실의 안전 규칙을 코드로 옮기는 핵심 도구

01 · np.any

하나라도 만족하면

True

조건을 만족하는 값이 하나라도 있으면 True

경고 발생 판정

02 · np.all

모두 만족해야

True

모든 값이 조건을 만족해야 True

전체 안전 판정

03 · 활용

비교 → 판정

두 단계 구성

비교 조건으로 True · False 배열을 만든 뒤

any·all로 최종 판정

any 한 줄이면 수만 개 중 문제 여부를 즉시 확인 — 1차 선별 검사

posco

any, all — 핵심 코드

하나라도 넘는지, 전부 안쪽인지 판정

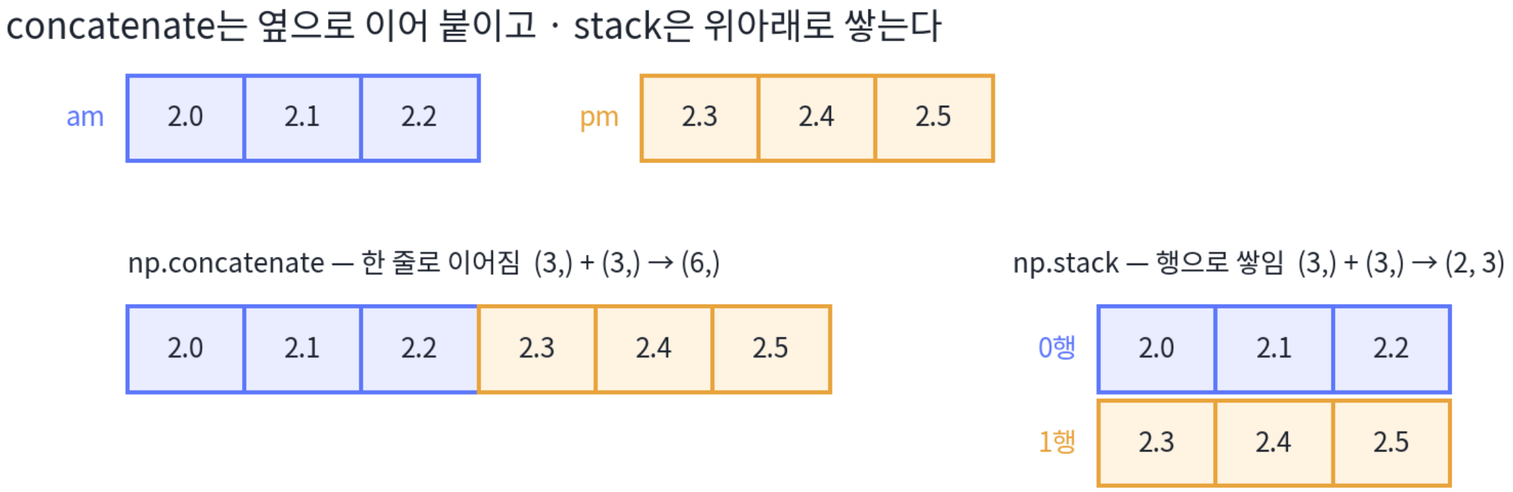
CODE

```
import numpy as np
vib = np.array([2.1, 2.3, 6.2, 2.0])
print(np.any(vib > 5))
# 출력: True
print(np.all(vib < 10))
# 출력: True
```

먼저 비교 조건으로 True·False 배열 생성 후 판정

센서 배열 합치기 — concatenate, stack

한 줄로 잇는 concatenate · 행으로 쌓아 차원을 늘리는 stack



센서마다 한 줄씩 쌓아 두면 「행 = 센서, 열 = 시점」인 표가 되고, 그때부터 axis로 방향을 정해 통계를 낼 수 있다

같은 센서의 오전·오후처럼 이어 붙일 때는 concatenate, 서로 다른 센서를 나란히 볼 때는 stack

센서마다 한 줄씩 쌓으면 「행 = 센서, 열 = 시점」인 표가 되고 그때부터 방향을 정해 통계를 낼 수 있음

쌓으려면 길이가 같아야 함 — 센서마다 측정 개수가 다르면 먼저 길이를 맞춰야 함

concatenate, stack — 핵심 코드

한 줄 연결과 2차원 쌓기 비교

CODE

```
import numpy as np
am = np.array([2.0, 2.1, 2.2])
pm = np.array([2.3, 2.4, 2.5])
print(np.concatenate([am, pm]))
# 출력: [2. 2.1 2.2 2.3 2.4 2.5]
print(np.stack([am, pm]).shape)
# 출력: (2, 3)
```

합칠 배열은 반드시 리스트로 묶어 넣어야 함

posco

데이터 준비 흐름

합치기 → 보정 → 라벨링 → 집계, 네 단계 흐름

STEP 1

합치기

concatenate · stack으로
흩어진 데이터를 모음

`concatenate · stack`

→

STEP 2

보정

clip으로 명백한 측정 오류를
정상 범위로 누름

`clip`

→

STEP 3

라벨링

where로 정상·경고·고장을
자동으로 매김

`np.where`

→

STEP 4

집계

unique로
전체 중 경고 개수를 요약

`np.unique`

순서가 바뀌면 — 보정 전 통계는 오류값에 휘둘림

03

종합 정리

오늘 배운 도구를 하나의 흐름으로 엮기

실습 5. 이상 시점 한 번에 정리하기

최대 시점 · 급변 구간 · 경고 비율 · 상위 세 시점을 한 흐름으로 정리

목표

오늘 배운 도구를 순서대로 이어 붙여 한 화면짜리 점검 결과 만들기

단계

① 시점 · 사이클타임 · 실린더압력 세 열을 불러오기

② 사이클타임이 가장 컸던 시점과 그때의 값 찾기

③ 이웃한 값의 변화량이 가장 컸던 구간 찾기

④ 사이클타임이 50 이상이거나 실린더압력이 150 아래인 시점을 경고로 보고 비율 구하기

⑤ 사이클타임이 큰 순서로 상위 세 시점을 뽑아 함께 출력

예상 결과

최대 50번 6170.0 · 급변 49→50 · 경고 비율 0.12 · 상위 세 시점 50 · 48 · 43

실습 5. 이상 시점 한 번에 정리하기 결과 해석

네 개의 답이 모두 뒤쪽 구간을 가리킴 — 설비 상태가 점점 나빠졌다는 뜻

01 · 결과가 한 곳을 가리킴

43번 이후에 몰림

최대 시점 50번, 급변 구간 49에서 50, 상위 세 시점 50 · 48 · 43. 네 지표가 모두 뒤쪽을 가리키면 우연이 아니라 상태 변화로 읽어야 함.

02 · 지표를 늘리면 늘어남

0.08에서 0.12로

사이클타임만 봤을 때 경고 이상은 8%였지만 실린더압력 조건을 더하면 12%. 43 · 45 · 46 · 47 · 48 · 50번 여섯 시점이 걸림. 지표 하나만 보면 놓치는 이상이 있음.

03 · 도구의 연결

따로 놀지 않음

위치 찾기 → 변화량 → 조건 판정 → 정렬. 오늘 배운 도구가 하나의 분석으로 이어짐. 함수를 외우는 것보다 이 흐름을 기억하는 편이 오래 남음.

다음 단원에서는 여기에 통계를 얹어 「정상은 어느 수준인가」부터 정의함

posco

실습 5. 이상 시점 한 번에 정리하기 정답

조건을 묶을 때는 각 조건을 반드시 괄호로 감싼다

CODE

```
import numpy as np
# 출처: 주조(다이캐스팅) 설비 실데이터(KAMP) 발췌
F = 'data/11_diecasting_series.csv'

def col(c):
    return np.genfromtxt(F, delimiter=',', skip_header=1, usecols=c)

cyc, ct, cyl = col(0), col(1), col(2)

i = ct.argmax()
print(int(cyc[i]), ct[i])                    # 50 6170.0

ch = np.abs(np.diff(ct)); j = ch.argmax()
print(int(cyc[j]), int(cyc[j + 1]))          # 49 50

warn = np.where((ct >= 50) | (cyl < 150), 1, 0)
print(round(warn.mean(), 2))                 # 0.12

top = np.argsort(ct)[::-1][:3]
print(cyc[top].astype(int))                  # [50 48 43]
```

사이클타임만 보면 경고 8%, 실린더압력을 함께 보면 12% — 지표 하나만으로는 놓침

CLOSING

오늘 배운 도구가 놓인 자리

위치를 찾고 · 변화를 재고 · 상태를 매기고 · 표로 묶는다

1

위치를 찾는다 — argmax · argsort

「얼마나」가 아니라 「언제」에 답해야 정비 이력을 찾아볼 수 있음

2

변화를 잰다 — diff · abs · clip

값보다 변화가 강한 신호. 재기 전에 명백한 오류값은 눌러 둠

3

상태를 매기고 묶는다 — where · unique · stack

숫자를 등급으로 바꾸고 센서를 표로 쌓는 것이 표 도구를 만나기 전 마지막 준비

다음 단원에서는 같은 일을 훨씬 짧은 코드로 하는 표 도구를 만남

CLOSING

posco

여기서 배운 것이 다음 단원에서는

도구가 바뀌어도 하는 일은 같음 — 훨씬 짧은 코드가 될 뿐

01 · 열 번호에서 열 이름으로

번호를 세지 않아도 됨

지금은 몇 번째 열인지 세어 넣었음. 다음 단원에서는 이름을 그대로 적음. 열 순서가 바뀌어도 안전해짐.

02 · 숫자만에서 글자까지

한 표에 함께

배열에는 숫자만 담을 수 있어 시각과 상태는 빼고 읽었음. 다음 단원의 표는 글자 열도 같은 표에 담음.

03 · 하나씩에서 한 번에

통계를 한 줄로

평균과 최대와 중앙값을 열마다 따로 구했음. 다음 단원에서는 숫자 열 전부에 대해 한 줄로 끝남.

오늘 익힌 흐름 — 자리를 찾고 변화를 재고 상태를 매긴다 — 은 그대로 이어짐